# HETEROGENEITY UNDER OBSERVABILITY

*An Acquisition-Cost Framework for Assessing Diversity as a Cyber-Physical Defense*

Response to DARPA-SN-27-17, Formal Learning and Abstractions for System Heterogeneity (FLASH)

**Submitted by AionSystem**

Technical POC: Sheldon K. Salmon · AionSystem@outlook.com · (315) 460-6977

**UNCLASSIFIED · November 2026**

## CONTENTS

EXECUTIVE SUMMARY . . . . . 2

§1. DOMAIN AND APPLICATION CLASS . . . . . 3

§2. THREAT ENVIRONMENT AND ADVERSARY CLASSES . . . . . 5

§3. STATE OF THE ART AND ITS STRUCTURAL FAILURE MODES . . . . . 8

§4. THE ACQUISITION BURDEN . . . . . 9

§5. THE INVARIANT SET, EXTENDED . . . . . 19

§6. DESIGN DEGREES OF FREEDOM . . . . . 20

§7. HETEROGENEITY METRIC: VARIANT INDEPENDENCE . . . . . 23

§8. VALIDATION OF EQUIVALENCE AND INVARIANTS . . . . . 30

§9. EVALUATION CRITERIA . . . . . 31

§10. SYNTHESIS AND ARCHITECTURE . . . . . 35

§11. REUSABLE FORMALIZATION LIBRARIES . . . . . 37

§12. ASSUMPTIONS, LIMITATIONS, AND FALSIFIERS . . . . . 38

ANNEX A. COVERAGE MAP . . . . . 43

ANNEX B. WHAT WOULD FALSIFY THIS RESPONSE . . . . . 44

REFERENCES . . . . . 44

POC AND FORMAT BLOCK . . . . . 45

## EXECUTIVE SUMMARY

FLASH proposes to make engineered systems more robust through heterogeneity: generate many provably correct, meaningfully different implementations of a specification, and use the diversity of failure modes and attack surfaces to reduce the durability and transferability of adversary knowledge. RFI 1(iv) asks respondents to name the key assumptions, limitations, and proposed evaluation methods for FLASH's application to specific domains. This response names the adversary's acquisition model as the assumption that decides heterogeneity's value, gives a method for determining which acquisition class applies to a program (§2.1), proposes a metric that predicts the adversary's burden under each class (§7), and gives an evaluation protocol that tests the prediction (§9).

The assumption that decides heterogeneity's value is the adversary's acquisition model. Two classes are structurally affected by heterogeneity; one is not. Under a class-attacking adversary ($A_1$), one who studies the family from public information and never accesses the deployment, heterogeneity raises the adversary's burden by the same structural multiplier as under capture-then-exploit, but at a lower per-variant unit cost. Under a capture-then-exploit adversary ($A_2$-hybrid), one who captures one deployed instance, studies its firmware and logic, and deploys exploits across the fleet, the burden is $[1 + H_{d,variant} \cdot (N-1)] \times (c_{capture} + c_{exploit})$. If $c_{capture} + c_{exploit}$ exceeds $c_{public}$, heterogeneity's absolute value is larger there. $A_2$-hybrid may be the dominant case for the RFI's own application examples, if forward-deployed platforms are capturable with extractable firmware, a per-platform question answered by the method in §2.1. Under a real-time adversary ($A_2$-pure), one who probes the deployed instance without study or persistent access, heterogeneity does not affect the burden. The RFI's thesis holds under $A_1$ and $A_2$-hybrid by the same structural multiplier. Under $A_2$-pure it does not apply to static or slow-dynamic heterogeneity (§4.3).

A program that builds heterogeneity without determining which class dominates cannot attribute resilience to heterogeneity. The main contribution is the acquisition-burden analysis (§2–§4), its metric (§7), and its evaluation protocol (§9). The response also addresses invariants, degrees of freedom, and assurance independence (§5, §6, §10.4, §10.5). The response supplies:

- A three-class adversary model by acquisition mode (§2), a method for determining which class applies to a program (§2.1), and the acquisition-cost burden that follows (§4), including a worked toy computation that checks the formula's arithmetic under stated assumptions (§4.5), and a spatial reading of the formula in five scenes: the homogeneous fleet, the heterogeneous fleet, the mixed fleet, the fleet that looked heterogeneous but was not, and the shared-hallway fleet whose capture is sublinear in N (§4.6).

- The dynamic extension: under dynamic heterogeneity, observability becomes predictability, and the requirement is unpredictability to the adversary with verifiability to the defender (§4.3).
- An extension of the invariant set to include degradation envelope, environmental drift, and calibration band, without which functional equivalence certifies the system that was tested, not the system that is deployed (§5).
- A distinction between apparent and genuine design degrees of freedom, semantic versus syntactic heterogeneity applied to a single DOF, with a test that separates them, illustrated through the COLREGs compliance case (§6).
- A heterogeneity metric, $H_{d,variant}$, computed as the variant fraction whose cheapest critical path is adversary-independent, at a declared analysis depth, with three analyst-judgment points resolved (§7.2) and the sources and directions of approximation error listed (§7.3).
- Six evaluation criteria with an adversarial protocol that measures time-to-compromise against a calibrated autonomous exploit-development agent under three adversary conditions, with outcome interpretation and statistical thresholds fixed before the run (§9).
- An independence requirement for co-generated assurance and explanation: architectural independence between synthesis and assurance, plus independent validation of the specification itself (§10.4).
- The explanation criterion: an explanation is faithful when a named, non-author checker can derive the assurance artifact's claim from it, and unfaithful when they cannot (§10.5).
- The anchor class is unmanned surface vessels and unmanned undersea vehicles (UUV/USV), defined by public regulatory documents, with its diversity and failure structure a subject the framework is built to test. §6.3 applies the framework's categories to the COLREGs. Evidence for the framework's central claim is §9's protocol, which has not been run.

## §1. DOMAIN AND APPLICATION CLASS

*[RFI 1(i)]*

**Domain.** Autonomous systems operating in contested environments: environments in which an adaptive adversary can observe, interact with, and attempt to compromise the deployed system.

**Target application class.** The unit of analysis is a design family (a set of functionally equivalent implementations generated from a single specification) deployed into an environment where each member is a realization, not a type. Fleets of autonomous robots, distributed edge networks, and autonomous platforms are members of this class. The framework is stated at the class level because the property under test is a class property: a single artifact cannot be heterogeneous with respect to itself.

**Anchor instance:** unmanned surface vessels (USVs) and unmanned undersea vehicles (UUVs). The anchor function for this response is autonomous maritime patrol: persistent presence in a

contested maritime area, sensor-fusion-based contact classification, and collision avoidance under COLREGs. Platforms are unmanned and mission-persistent, operate without continuous operator control, rely on onboard contact classification and COLREGs-governed avoidance, and use intermittent encrypted command-and-control links, in contested coastal and open-ocean waters within physical reach of adversary vessels and aircraft.

The class is anchored here because the maritime autonomy domain exhibits the properties the framework tests for, as observable from public regulatory documents and published engineering practice. UUVs and USVs are forward-deployed by physical necessity: a vessel that operates in the maritime environment must be present in it. The design family plausibly exhibits the variation the framework tests for: hull form, propulsion, sensor suite, communications, and autonomy stack vary across vendors and documented platform classes. Whether that variation moves critical failure nodes (the §6.2 test) is a question the framework is built to answer. The contested environment is physical and adversarial, not cyber-only.

The regulatory dimension compounds the technical one. A USV must demonstrate compliance with the International Regulations for Preventing Collisions at Sea (COLREGs), which require every vessel to maintain “a proper look-out by sight and hearing as well as by all available means appropriate in the prevailing circumstances and conditions” (Rule 5). That specification was written for a human on a bridge, and it is instantiated in sensor and autonomy stacks whose public compliance-adjudication history is sparse (§12, L4). The COLREGs case is worked in §6.3.

**Basis of the anchor.** The anchor is named from public regulatory documents (COLREGs, 33 CFR Part 83) and general industry practice. The basis is declared in §12, L4.

**Three conditions for heterogeneity to pay.** The class has a property that makes the question testable:

(a) **Genuine degrees of freedom.** The family has design variation that provably preserves the required behavior (§6).

(b) **Adversary acquisition model.** The adversary is not fully state-observant through real-time probing without capture or study (§2, §4). Physical visibility of the deployed platform does not by itself imply the state-observability required for $A_2$-pure.

(c) **Verification economy.** Per-variant verification cost does not exceed the value of the diversity, given the adversary class (§4). Criterion 6 of §9.1 records the costs.

Condition (c) is where FLASH’s synthesis thrusts and this response’s verification thrust meet: FLASH’s thrust (ii) generates per-variant equivalence guarantees, and this response asks whether those guarantees buy enough against the adversary class that actually dominates. Condition (b) decides whether heterogeneity pays. RFI 1(iv) asks for key assumptions, and this is the one this response names.

## §2. THREAT ENVIRONMENT AND ADVERSARY CLASSES

*[RFI 1(ii), 1(iv)]*

**National-security rationale.** Contested autonomous systems concentrate national-security value because they place decision and effect at the edge: at the point of engagement, with hardware that degrades and software that is updated. The threat is not a single exploit but an adversary who can choose when and how to specialize. Capture of unmanned maritime vehicles by adversary forces is documented: on 15 December 2016 a Chinese Navy vessel seized a U.S. Navy underwater glider about 50 nautical miles northwest of Subic Bay and returned it on 20 December [R1, R2]. The event shows that a deployed maritime vehicle can come into adversary hands. Cyber-intrusion and supply-chain paths to the same artifacts are per-platform questions addressed by the method in §2.1.

**Three adversary classes, separated by acquisition mode.** The RFI refers to instance-specific adversary knowledge without separating how it is acquired. Acquisition mode gives three classes, and heterogeneity's value differs across them.

| | $A_1$: class-attacking | $A_2$-hybrid: capture-then-exploit | $A_2$-pure: real-time |
|---|---|---|---|
| **What the adversary acquires** | Knowledge of the design family from public or purchased information | A deployed instance (by physical capture or by cyber intrusion into the platform) | Transient interaction with the deployed instance; no capture, no study |
| **Means** | Mass exploitation, common-mode vulnerabilities, published CVE reuse | Firmware/logic extraction → exploit development → fleet-wide deployment; supply-chain insertion to reach the deployment | Real-time probing, side-channel use, jamming, short-lived exploits |
| **Acquisition cost, homogeneous family ($H_{d,variant} = 0$)** | $1\times c_{public}$ | $c_{capture} + c_{exploit}$ → fleet compromised | 1× per-engagement cost |
| **Acquisition cost, heterogeneous family ($H_{d,variant} > 0$)** | $[1 + H_{d,variant}\cdot(N-1)] \times c_{public}$ | $[1 + H_{d,variant}\cdot(N-1)] \times (c_{capture} + c_{exploit})$ | 1× per-engagement cost |
| **Heterogeneity's effect** | Structural multiplier same as $A_2$-hybrid; lower per-variant unit cost | Structural multiplier same as $A_1$; higher per-variant unit cost | None: heterogeneity does not touch this cost |

The classes are a taxonomy, not an exclusive typing. An adversary can pair $A_1$ preparation (public study) with $A_2$-pure engagement (real-time probe). The burden calculation uses the class that dominates the adversary's work.

**Why the distinction matters.** The dominant attack on a forward-deployed platform may be capture-then-exploit: physically take one, study it, exploit the fleet. That is $A_2$-hybrid, and it is the case where heterogeneity's absolute value may be largest, because heterogeneity turns one capture into one variant's worth of exploit, forcing the adversary to repeat the capture-and-exploit cycle per variant.

**When the $A_2$-hybrid formula holds.** The formula $[1 + H_{d,variant} \cdot (N-1)] \times (c_{capture} + c_{exploit})$ assumes:

(i) **No missed transfer**: that the §7.2 test has identified every transfer between variants at or above $\theta$, so that capturing one variant in the independent set gives no material advantage toward another.

(ii) **Post-test, post-reduction $H_d$**: that the $H_{d,variant}$ in the formula is the value produced by §7.2's full procedure, not an apparent value.

(iii) **Cluster structure**: the formula assumes the non-independent $(1 - H_{d,variant})$ variants form one transfer cluster. If they form $k_{clusters}$ transfer-equivalence classes, the cycle count is $H_{d,variant} \cdot N + k_{clusters}$ ($k_{clusters} = 0$ when no variant is non-independent), and $H_{d,variant}$ as a fraction does not capture $k_{clusters}$. §4.5 gives the bound.

(iv) **Topology-adjusted capture term**: that the number of captures is $\max(1, \lceil \text{cycles} / k_{cap} \rceil)$ for the declared deployment topology (§4.6 Scene 5).

If a shared vulnerability class exists (any of the eleven mechanisms listed in §4.6 Scene 4), then even variants with distinct critical failure nodes may share exploitation techniques, and the formula understates the adversary's capability: the real cost is closer to the homogeneous case. The $H_{d,variant}$ in this formula is the post-minimum-path value defined in §7.2, the variant fraction whose cheapest critical path is adversary-independent, not the node ratio computed by decomposing to critical nodes alone. §4.5 works the formula through a toy computation; §4.6 shows five fleet configurations in scene form, including the case where an apparently-heterogeneous fleet has $H_{d,variant} = 0$ because its variants share a hidden centralization.

**$A_2$-hybrid may be the RFI's dominant case.** Forward-deployed hardware is physically exposed such that capture is possible in principle. Whether capture yields actionable firmware access, rather than a destroyed or protected device, depends on the platform's tamper resistance, its operational security posture, and the adversary's resources. The anti-tamper enclosures and post-mission forensics listed in §3 are the current defenses; their effectiveness determines

whether $A_2$-hybrid is dominant for any specific program. The RFI's own application examples (autonomous tactical platforms, fleets of autonomous robots, edge networks) are exposed to capture by physical necessity, but the dominance of $A_2$-hybrid for any of them is a per-program hypothesis. The example list also includes secure communication pipelines, industrial control systems, HPC clusters, and cybersecurity applications, to which the analysis applies differently: these are less exposed to physical capture but more exposed to supply-chain and remote-intrusion $A_2$-hybrid paths.

$A_2$-pure is the case where static heterogeneity does not change the burden (§4.3 covers dynamic heterogeneity).

**The threat environment.** All three classes are live. The class that dominates for a given program decides heterogeneity's value, so the acquisition class is the key assumption this response names for RFI 1(iv). §2.1 gives the method for determining it.

### 2.1 Determining which class applies

The dominant class for a platform and program is determined by assessment, not by the §9 experiment. The proposed method has four inputs.

- **Capture exposure:** the likelihood of physical loss or seizure in the operating area, from operational history and public threat reporting.
- **Extractability:** tamper resistance, storage encryption, key management, and the firmware-extraction effort demonstrated in a red-team capture exercise on the platform class.
- **Remote paths:** remote-intrusion and supply-chain routes to the same artifacts.
- **Live observability:** whether an adversary probing the live platform can observe the state needed to exploit it.

The output is, for each platform, a ranked list of acquisition classes with the evidence and the cost ranges assumed for $c_{capture}$, $c_{exploit}$, and $c_{public}$. $A_2$-hybrid is treated as dominant when the extraction exercise succeeds at a cost below the value of compromising the fleet. $A_2$-pure is treated as dominant when live observation suffices and extraction fails. The §9 protocol tests the burden under each class and does not determine which class applies.

## §3. STATE OF THE ART AND ITS STRUCTURAL FAILURE MODES

*[RFI 1(iii)]*

Each current defense against adversarial compromise of autonomous systems has a specific structural failure mode. The characterization below is a reading of the field (§12, L5).

| Technique | What it does | Structural failure |
|---|---|---|
| Redundancy (N-modular, voting) | Tolerates component failure by replication | Correlated failure. Identical replicas share identical failure modes. Redundancy is defeated by anything the replicas have in common. |
| Design diversity | Independent implementations of the same function | Specification correlation. Diverse implementations of one spec share the spec's blind spots. Diversity in code, identity in requirements. Knight and Leveson [R3] measured coincident failures across independently written versions; Littlewood and Miller [R4] model them. |
| Formal verification | Proves properties of a model | Model–reality mismatch. The proof is of the model. Field failures include what the model omitted (environment, wear, calibration drift) alongside proof error. |
| Red-teaming / penetration testing | Finds exploitable paths empirically | Coverage non-transfer. A finding is instance-specific. Testing variant *n* tells you about variant *n*. Cost scales with the number of variants. |
| Moving target defense | Randomizes attack surface over time | Observable transition. If the adversary can observe the transition, the defense relocates the target rather than hiding it. Designed to defeat $A_1$-style attacks; the analysis in §4.3 shows the limits against $A_2$-pure. |

**The common assumption.** Each technique depends on an assumption about independence or observability: redundancy assumes replicas are independent; diversity assumes specs are complete; verification assumes models are faithful; testing assumes coverage transfers; moving target defense assumes transitions are unobserved.

**Anchor-specific context.** For USV/UUV systems, current practice includes encrypted command-and-control links, anti-tamper enclosures, and post-mission forensics. Anti-tamper enclosures and post-mission forensics are the current defenses that determine whether an $A_2$-hybrid capture yields actionable study access; their effectiveness is the $c_{capture}$-determining factor for any specific platform. Supply-chain provenance enforcement and fleet-wide trust-anchor reissuance conventions are two areas where practice is currently limited.

# §4. THE ACQUISITION BURDEN

*[RFI 1(iv)]*

**What FLASH claims.** Generated instances may exhibit meaningful variation in failure modes and attack surfaces while preserving specified system properties, and that variation "could reduce the durability and transferability of instance-specific adversary knowledge" (RFI p.2). The claim holds where the adversary must acquire knowledge of the deployed instance; it fails where the adversary does not. The analysis below makes the distinction precise.

**The finding.** Heterogeneity raises the adversary's acquisition cost against $A_1$ and $A_2$-hybrid by the same structural multiplier, at different per-variant unit costs. It leaves that cost unchanged against $A_2$-pure. Transfer and acquisition are different quantities, and FLASH's thesis is stated in terms of the first while its practical value depends on the second.

## 4.1 The two-sided burden

The defender's burden and the adversary's burden use different units (verification-hour versus exploit-development-hour), amortize differently (verification is incurred once per variant set at design time; exploit development against an unchanged variant is incurred once, because the exploit persists), and automate differently (thrust (ii)/(iii) are aimed specifically at making verification cheap). What matters is which side's burden grows with N, and how the absolute burdens compare where the units allow.

**Defender's burden:** N × (per-variant verification cost) plus the cost of computing $H_{d,variant}$. Verification is a property of the artifact and must be done for each variant. No variant inherits another's assurance. This is N growth regardless of adversary class.

The $H_{d,variant}$ computation is pairwise: §7.2's independence test and minimum-path reduction compare each variant-specific node against every other variant, so the computation is $O(N^2)$ in the worst case. Each comparison estimates the adversary effort to transfer an exploit between two variants (J-2, J-3), a cost of the same kind as the per-variant red-team testing listed in §3. The defender's total cost is N × per-variant verification plus up to $N^2$ pairwise estimates. Whether the computation is amortizable across variant families or reducible by heuristics is not resolved here (§12, L8).

Both defender costs are incurred once per variant set, and so is the adversary's, because an exploit against an unchanged variant persists. Repeated attacks add to the adversary's cost only where its knowledge is invalidated (a replaced or patched variant, a closed capture channel), and the defender then pays to verify the replacement; the comparison is per variant set, not per attack.

**Adversary's burden, by class:**

**$A_1$: class-attacking.** The adversary must solve each variant whose cheapest critical path is not inferable from public information about other variants. Cost is $[1 + H_{d,variant} \cdot (N-1)] \times c_{public}$, where $c_{public}$ is the per-variant public-analysis cost. Structurally identical to $A_2$-hybrid with a lower per-variant unit cost.

**$A_2$-hybrid: capture-then-exploit.** The adversary must independently capture and exploit each variant whose cheapest critical path is adversary-independent. Against a homogeneous family ($H_{d,variant} = 0$), one capture-and-exploit cycle compromises the fleet: total cost $c_{capture} + c_{exploit}$. Against a heterogeneous family ($H_{d,variant} > 0$), the adversary must repeat the cycle on the $H_{d,variant}$ fraction of variants whose cheapest path does not transfer; the $(1-H_{d,variant})$ fraction can be compromised from a single study. Cost is $[1 + H_{d,variant} \cdot (N-1)] \times (c_{capture} + c_{exploit})$, subject to the four assumptions in §2: no missed transfer, post-test $H_d$, cluster structure, and the topology-adjusted capture term. The formula is the continuous form of the cycle count; the discrete count is $\lceil 1 + H_{d,variant} \cdot (N-1) \rceil$ (§4.5). §4.5 works the formula through a toy computation, and §4.6 shows five fleet configurations in scene form.

**$A_2$-pure: real-time.** The adversary probes the deployed instance and exploits it before losing access. Cost is 1 × (per-engagement cost). Heterogeneity does not change this cost, because the engagement is fresh regardless.

**The asymmetry.** Under both $A_1$ and $A_2$-hybrid, the defender grows ×N (plus $H_d$ computation) and the adversary grows $\times[1 + H_{d,variant} \cdot (N-1)]$, a factor of N at $H_{d,variant} = 1$. The structural multiplier is the same in both classes. The difference is the per-variant unit cost: $c_{capture} + c_{exploit}$ for $A_2$-hybrid versus $c_{public}$ for $A_1$. If $c_{capture} + c_{exploit}$ exceeds $c_{public}$, heterogeneity's absolute value is larger under $A_2$-hybrid; the response has no cost data for either quantity. If the $H_d$ computation grows as $N^2$, the defender's cost grows faster than the adversary's. Under $A_2$-pure, for static heterogeneity, the defender grows ×N and the adversary's cost does not grow.

Whether heterogeneity pays in absolute terms depends on how the per-variant exploit cost compares to the per-variant verification cost. If exploit development is expensive per variant relative to verification, heterogeneity is a strong bet under $A_1$ and $A_2$-hybrid. If exploit development is cheap (for instance because variants share a common vulnerability class despite variation at the implementation level), heterogeneity's structural advantage is not realized. The §7 metric measures the structural quantity ($H_{d,variant}$); the §9 protocol measures the behavioral

one (C(A, F)); their correspondence is the hypothesis under test.

### 4.2 How heterogeneity does mitigate *[RFI 1(iv)]*

**Vulnerabilities.** Against $A_1$ and $A_2$-hybrid, a variant-specific defect is not a fleet defect. Blast radius shrinks from fleet to variant, provided the variant fraction with independent cheapest paths is high (§7.2).

**Common-mode failures.** Diversity attacks correlated failure directly, and correlated failure is a significant risk in systems built from common components. The §3 qualification holds: implementation diversity does not address specification diversity.

**Common fault modes.** A fault mode shared across a family is knowledge that transfers. Under $A_1$ and $A_2$-hybrid, heterogeneity breaks transfer where $H_{d,variant} > 0$. Under $A_2$-pure, it does not.

**Attack pathways not addressed by current defenses.** The pathways current defenses structurally cannot reach are the ones that assume per-instance verification: supply-chain insertion into a shared component, a published flaw in a common stack, an exploit developed against one captured platform and reused across a fleet. Heterogeneity reaches these to the extent the shared component or stack is varied across the family. A component common to all variants is a hidden centralization (§4.6, Scene 4) and is not reached.

### 4.3 Dynamic heterogeneity

The analysis above treats heterogeneity as a static property of a variant family. The RFI also asks about dynamic heterogeneity: variation that changes within a deployment or across engagements. The burden structure survives the shift, with a modification.

Under dynamic heterogeneity, the acquisition condition becomes a predictability condition. The adversary's problem shifts from "which variant am I facing" to "which distribution am I facing, and can I predict the sequence."

**Predictable dynamics.** If the sequence is determined by observable rules (a rotation schedule, a selection policy, a public configuration), the adversary specializes against the sequence. This is the $A_2$-hybrid failure at fleet scale.

**Unpredictable dynamics.** If the sequence is randomized over a space the adversary cannot enumerate, the adversary must specialize against the distribution. This is the $A_1$ case with an additional cost: the adversary cannot be certain which variant family is deployed.

**The finding.** Dynamic heterogeneity helps under $A_1$ and under $A_2$-hybrid-when-unpredictable. It does not help under $A_2$-hybrid-when-predictable or $A_2$-pure. For $A_2$-pure, the "not at all" claim is scoped to static and slow-dynamic heterogeneity; if the dynamics change faster than an engagement, the probe-loop faces a moving target, which is a different mechanism from the

burden analysis above, and this section does not address it. Under $A_2$-hybrid, unpredictability requires that the selection mechanism's entropy source and policy cannot be recovered from the captured instance. A seed or policy stored on the platform is readable by capture-and-study, and the dynamics are then predictable.

**The requirement that makes dynamic heterogeneity worth its cost:** the dynamics must be unpredictable to the adversary but verifiable by the defender. A dynamic defense that is predictable to the adversary is a static defense with additional verification cost. A dynamic defense that is unpredictable to the defender is an unverified system in motion.

This extends §4's central finding. The acquisition burden $[1 + H_{d,variant} \cdot (N-1)]$ applies to static heterogeneity and to predictable-dynamic heterogeneity (the adversary can read both). For unpredictable-dynamic heterogeneity, the adversary cannot read the sequence, and the burden shifts toward the $A_1$ case (the adversary must solve the distribution).

### 4.4 Where FLASH's thrusts map

Thrust (i) supplies the degrees of freedom (§6); thrust (ii) supplies per-variant equivalence guarantees (§8); thrust (iii) supplies assurance and explanation collateral (§10). None of the three states which adversary class the cost structure assumes. Thrust (ii)'s functional-equivalence guarantee is silent on the acquisition cost, which is the term that decides whether the guarantee pays.

### 4.5 Worked consistency check: the toy fleet

The argument above is analytic. It can be checked against a toy case before any empirical protocol runs. The toy computation is a structural check. Validation against reality is §9's job.

**Setup.** 100 deployed instances. Adversary is $A_2$-hybrid: capture one instance, study, exploit, deploy. The per-variant capture cost is $c_{capture}$; the per-variant exploit-development cost is $c_{exploit}$. $c_{capture}$ and $c_{exploit}$ are assumed independent across variants, the caveat from §2 and the boundary established below. The toy assumes one cheapest critical path per variant, which is the one-lock scope the scenes reflect. Real variants may have several critical nodes, and the adversary takes the cheapest; §7.2's minimum-path reduction is what converts a node count into the per-variant cost the formula uses. Without that reduction the quantity is $H_{d,node}$, which is not the formula's input (see §7.2, §7.3, and §4.6 Scene 3's scope note).

**Homogeneous case ($N = 1$, $H_{d,variant} = 0$):**

- One capture: $c_{capture}$
- One exploit development: $c_{exploit}$
- One deployment across all 100: fleet compromised

- Total cost to the adversary: $c_{capture} + c_{exploit}$

**Heterogeneous case (N = 10 variants, 10 instances per variant, $H_{d,variant} = 1$):**

- Variant 1: $c_{capture} + c_{exploit} \rightarrow$ 10 instances compromised
- Variant 2: $c_{capture} + c_{exploit} \rightarrow$ 20 instances compromised
- … repeat for each of the 10 variants
- Total cost to the adversary: $10 \cdot c_{capture} + 10 \cdot c_{exploit} = 10 \cdot (c_{capture} + c_{exploit})$

**Heterogeneous case (N = 10 variants, $H_{d,variant} = 0.3$):**

- 3 variants require independent capture-and-exploit cycles: $3 \cdot (c_{capture} + c_{exploit})$
- 7 variants are compromised from one variant's study: $1 \cdot (c_{capture} + c_{exploit})$
- Total cost: $4 \cdot (c_{capture} + c_{exploit})$

The formula gives $1 + 0.3 \cdot (10-1) = 3.7$. With $H_{d,variant} \cdot N$ independent variants and one transfer cluster for the rest, the exact count is $H_{d,variant} \cdot N + 1$ cycles for $0 < H_{d,variant} < 1$ (4 here), N cycles at $H_{d,variant} = 1$, and 1 cycle at $H_{d,variant} = 0$. The linear formula equals the exact count at both endpoints and is below it by $H_{d,variant}$ cycles in between; its ceiling equals the exact count.

**What the check confirms.** The formula in §4.1 produces values within one cycle of the discrete count for the toy case. The adversary's burden under $A_2$-hybrid rises with $H_{d,variant}$ as the §4 argument claims. The formula is arithmetically consistent under the stated assumptions.

**What the check does not confirm.**

The formula holds only when the §7.2 test has found every transfer between variants. If variants share a common software stack, common supply chain, common physical weakness, or a shared low-entropy generator, then a single capture-and-study yields exploitation insight for more than one variant, and the formula understates the adversary's capability. The real cost is closer to the homogeneous case whenever a shared centralization exists. §7.2's minimum-path reduction and independence test are what determine whether independence holds in a given family.

The formula assumes each variant must be captured separately. Where variants are co-located (a single base, shared maintenance facility, fleet-in-port) or channel-connected (variants interoperate by design: §5's interoperability invariant), a single capture may reach $k_{cap}$ variants. The number of captures is then $\max(1, \lceil \text{cycles} / k_{cap} \rceil)$, and $c_{exploit}$ is still paid per cycle. This depends on deployment topology and is not a departure from $A_2$-hybrid. §4.6 Scene 5 works this case in scene form; the formula's cycle count is unchanged, but the capture term is the topology-adjusted one.

The formula assumes the non-independent $(1 - H_{d,variant})$ fraction of variants forms a single transfer cluster. If they form $k_{clusters} > 1$ transfer-equivalence classes, the cycle count is $H_{d,variant} \cdot N + k_{clusters}$, an increase of $(k_{clusters} - 1)$ over the formula. The increase grows with N: with $H_{d,variant} = 0$ and variants clustered in pairs, the count is N/2 against the formula's 1. $H_{d,variant}$ as a fraction does not capture the cluster structure, so reports give $k_{clusters}$ alongside $H_{d,variant}$. §2's fourth assumption states this; §7.2's partial-sharing definition provides the boundary for the metric itself.

The formula says nothing about what $H_{d,variant}$ is for a real fleet. $H_{d,variant}$ is measurable only through §7's decomposition and reduction; the toy case uses an idealized $H_{d,variant} = 1$ to check the formula's behavior.

**The check's finding for the document.** The formula in §4.1 is arithmetically consistent under the four stated assumptions (no missed transfer, post-test $H_d$, cluster structure, topology-adjusted capture term). The argument that heterogeneity raises the adversary's burden under $A_2$-hybrid is arithmetically consistent under the stated assumptions. Whether the argument holds for a real fleet is what §9's protocol tests. The independence assumption is where the formula can break, and it breaks toward the homogeneous case, meaning the adversary's burden is lower than the formula predicts whenever independence fails. Whether independence holds is a per-family question, and §7.2's test is how it is answered. §4.6 gives the same computation as a set of scenes.

### 4.6 Spatial reading of the burden formulas

The burden formulas in §4.1 can also be read as scenes.

The first four scenes are the same building. A building has 100 doors. Each door has a lock. In the capture-then-exploit framing of $A_2$-hybrid, the "lock" is the exploitable path on a captured instance: the firmware, the configuration, the protocol, the exploitable logic. The adversary's goal is every door open.

**Scene 1: homogeneous fleet ($H_{d,variant} = 0$).** All 100 doors have the same lock. The adversary picks the lock once, studies the mechanism, and the master key opens every door. The building is compromised for the cost of one lock pick: $c_{capture} + c_{exploit}$.

**Scene 2: heterogeneous fleet ($H_{d,variant} = 1$).** All 100 doors have different locks. The adversary picks door 1's lock; the technique does not open door 2. Door 2 requires a new pick. Door 3 requires another. The building is compromised for the cost of 100 distinct lock picks: 100 ×

$(c_{capture} + c_{exploit})$.

Scene 3: mixed fleet ($H_{d,variant} = 0.3$, N = 10 variants of 10 doors each). Ten door types, 10 doors each. Three types have unique locks (30 doors, three lock mechanisms). Seven types share one lock (70 doors). The adversary picks four locks total (one shared, three unique) and opens all 100 doors: $4 \times (c_{capture} + c_{exploit})$, matching $\lceil 1 + 0.3 \cdot (10-1) \rceil \times (c_{capture} + c_{exploit})$, the ceiling of 3.7.

**Scene 3's scope notes.** The scene assumes each door has one lock: one cheapest critical path per variant. Real variants have several critical nodes, and the adversary takes the cheapest. This is the one-lock scope the formula uses. If a variant has 9 variant-specific nodes and 1 shared node, and the shared node is the cheapest path, then the adversary picks the shared lock (one pick, all variants open) even though the variant's node count is dominated by variant-specific entries. §7.2's minimum-path reduction is what converts the variant's node profile into the per-variant cost the formula uses, and §7.3's worked example shows why the reduction matters. The scene's arithmetic is correct only under the one-lock scope. The scene also assumes a single transfer cluster (the seven shared-lock door types share one lock mechanism); multiple clusters of shared locks require separate studies and increase the cycle count.

**Scene 4: the fleet that looked heterogeneous but was not.** A fleet of 100 ships, each with a different hull form, a different sensor suite, a different vendor on the title plate. To an observer, the fleet looks diverse: $H_{d,variant}$ appears high. The adversary captures one ship, studies its firmware, develops an exploit. Deploys the exploit against the fleet. Every door opens. The hulls were different; the firmware was the same. The variants were variants in appearance only.

This is the failure mode Scenes 1–3 do not show, because those three assume the variants are actually heterogeneous. Scene 4 shows what happens when they are not. The apparent distribution was a hidden centralization.

**Eleven mechanisms of hidden centralization.** Scene 4's concrete case is a shared firmware stack, but the class of hidden centralizations is broader. The adversary's knowledge transfers across variants when the variants share:

(a) a common software stack;

(b) a common supply chain or common components sourced from a single vendor;

(c) a common physical weakness in a material or manufacturing process;

(d) a shared generator whose low-entropy or predictable choices point the adversary at the family's failure surface without requiring a capture;

(e) a shared specification or protocol, as in §3's specification correlation and §5's interoperability invariant requiring common interfaces;

(f) shared trust anchors, keys, or update/management channels;

(g) a shared toolchain;

(h) a shared environmental dependency such as GNSS;

(i) a shared deployed perception or classification model, including its weights or training data;

(j) shared in-envelope behavior: the invariant set (§5) requires equivalent behavior inside the envelope, so an exploit that acts through specified behavior, such as a spoofed contact that induces a compliant maneuver, applies to every variant;

(k) shared sensing phenomenology, such as radar or acoustic spoofing and jamming at the physical layer.

Mechanism (d) is a property of the generation process, not of the variants themselves: if a generative model makes consistent choices across a family, an adversary who learns the model's bias gains discovery advantage against all variants. Mechanism (e) is what §3's specification correlation names as the structural failure of design diversity. Mechanism (f) is what §3's absence of fleet-wide reissuance conventions leaves unaddressed. §7.2's independence test (material advantage of at least θ) captures mechanisms (a)–(c), (f)–(i), and (k) at the node level. Mechanisms (e) and (j) are specification-level and are not nodes, so $H_{d,variant}$ does not price them (§12, L10). Mechanism (d) gives discovery advantage without a capture, applies to $A_1$ as well, and requires a family-level check outside the node-level test; §7.2 states that check.

This is what §7.2's minimum-path reduction and independence test are designed to detect, and why §4.1's formula requires the post-reduction $H_{d,variant}$. The test does not ask whether the variants look different. It asks whether the exploitation technique developed against one variant transfers to another, or whether knowledge of one variant's design provides discovery advantage against another. If either transfers, the fleet has a hidden centralization, and its $H_{d,variant}$ is 0 in the sense the formula cares about, regardless of how different the variants appear. Plugging an apparent $H_{d,variant}$ (computed without the test) into §4.1's formula overstates the adversary's burden by up to a factor of N.

**Scene 5: the shared hallway.** The doors above are treated as separate targets. But a building can be laid out so that one door opens into a hallway connecting to every room. The adversary captures one door (pays $c_{capture}$), enters the hallway, and from there can study the variants reachable from it without a separate physical capture, since the channel between variants carries the adversary's reach. The same happens when variants are co-located (a single base, a shared maintenance facility, a fleet-in-port) or channel-connected by design (variants interoperate by §5's interoperability invariant, and a compromised variant can reach its siblings through the operational channel). The number of captures needed is $\max(1, \lceil \text{cycles} / k_{cap} \rceil)$, where $k_{cap}$ is the

average number of variants reachable per capture. Total cost is captures × $c_{capture}$ + cycles × $c_{exploit}$. The formula's cycle count is unchanged; the capture term is the topology-adjusted one. This is $A_2$-hybrid; the topology affects the capture cost, not the class. A program whose deployment topology gives $k_{cap}$ variants per capture has a weaker heterogeneity defense than the same fleet deployed with $k_{cap} = 1$, even at identical $H_{d,variant}$.

**What the five scenes show.** The $A_2$-hybrid burden formula, $[1 + H_{d,variant} \cdot (N-1)] \times (c_{capture} + c_{exploit})$, is the cost of picking $1 + H_{d,variant} \cdot (N-1)$ distinct locks, times the cost per pick, under four conditions: (i) each variant's cost is its cheapest critical path (Scene 3 scope note); (ii) $H_{d,variant}$ is the post-minimum-path value (Scenes 3–4); (iii) $c_{capture}$ and $c_{exploit}$ are independent across variants (Scene 4); (iv) captures are counted with the topology adjustment of Scene 5. Scenes 1–3 confirm the formula's behavior. Scene 4 confirms why the minimum-path reduction and independence test must precede the computation. Scene 5 confirms that the capture term depends on deployment topology, not only on per-variant capture difficulty.

The two other adversary classes each have their own scene. The five scenes above are all $A_2$-hybrid.

$A_2$-pure is a peephole scene. The adversary walks up to a door and looks through the peephole. If the door is vulnerable in that moment, it opens. If not, the adversary walks away. Each approach costs the same. The scene has 100 identical doors, and it does not matter if they have 100 different locks, because the adversary never picks a lock, only looks through peepholes. Heterogeneity does not change the scene.

$A_1$ is a blueprint scene. The adversary has blueprints. Each blueprint shows one lock type. The adversary studies the blueprints without approaching the building. If the variants share blueprints (i.e., if the fraction $(1 - H_{d,variant})$ of variants is buildable from blueprints the adversary has already studied), then the adversary studies $1 + H_{d,variant} \cdot (N-1)$ distinct blueprints. N variants require N distinct blueprints at $H_{d,variant} = 1$, and 1 blueprint at $H_{d,variant} = 0$. The $A_1$ burden and the $A_2$-hybrid burden share the same structural multiplier; the difference is the per-blueprint cost ($c_{public}$) versus the per-cycle cost ($c_{capture} + c_{exploit}$).

**What the scenes do not show.** The scenes are missing four elements:

**The defender.** The scenes are adversary-eye only. The defender's burden (N × per-variant verification plus the $H_{d,variant}$ computation) is design-time and off-screen. §9.1 criterion 6 records it.

**Real numbers.** The scenes use “lock picks” as their unit. They cannot tell you whether a lock pick takes an hour or a month. Whether the adversary’s 100 lock picks is prohibitive or trivial depends on real $c_{capture}$ and $c_{exploit}$ values, and neither the scenes nor the rest of the document has those. §9’s protocol would measure them.

**Independence failure.** Scenes 1–3 assume the locks are independent: picking one gives no advantage on the next. Scene 4 shows what happens when they are not. Whether a real fleet’s locks are independent is what §7.2’s minimum-path reduction and independence test determine, and the test requires a real variant family to run against. Published multiversion experiments with recorded coincident failures (Knight and Leveson [R3]) are candidates for a first application (§7.5).

**The definition of success.** The scenes and the formula price “fleet compromised”: every door open. The adversary’s actual success criterion may be less: a majority of variants, a critical subset, or mission-kill. When the goal is less than every door, the effective N in the formula is the number of variants the adversary needs to reach, not the total fleet count. A single-variant deployment (deployment collapse: only one variant fielded for procurement, logistics, or mission reasons) has N = 1, and the formula degrades to the homogeneous case for that fielded subset regardless of the design-time $H_{d,variant}$. Heterogeneity is a property of the deployed fleet, not the designed family. A program that builds a highly diverse family but deploys one variant has heterogeneous design and homogeneous deployment.

**The scenes’ final implication.** The structural argument (heterogeneity raises the adversary’s burden under $A_1$ and $A_2$-hybrid when $H_{d,variant}$ is high and the capture term is topology-adjusted) follows from the formula and Scenes 1–3. The practical claim (heterogeneity pays for a specific fleet, against a specific adversary, at acceptable cost) requires the defender’s burden, real numbers, independence confirmation, and the topology-adjusted capture term. §9’s protocol measures all four. Scene 4 shows why the third (independence confirmation) is a precondition for interpreting the formula. Scene 5 shows why the fourth (topology) is a parameter of the formula, not a given.

## §5. THE INVARIANT SET, EXTENDED

*[RFI 1(v)]*

The RFI asks for properties that "must remain invariant across generated alternatives" and names six categories. The six do not name the invariants that determine whether functional equivalence survives service life.

| RFI invariant | What it must cover for this domain | Status |
|---|---|---|
| Functional | Input–output map preserved across variants: the same classification output and avoidance command for the same sensor input | Stated; requires model scope declaration (§8) |
| Safety | Hazard set unreachable under specified conditions: collision, grounding, loss of control | Stated; hazard set must include degradation states |
| Security | Adversary cannot achieve effect within threat model: command injection, loss or diversion of the platform, falsified contact data | Stated; threat model is the §2 three-class model |
| Timing | Deadlines and synchronization preserved: sensor-to-avoidance-command latency, fusion-cycle deadline | Stated; timing degrades under thermal and load variation |
| Interoperability | Variants operate within a common fleet | Stated; strongest argument for cross-variant equivalence; also the channel that Scene 5 uses |
| Performance | Throughput, latency, accuracy within bounds: classification accuracy at declared range and sea state; speed and endurance | Stated; bounds shift with wear |
| Degradation envelope | Behavior preserved across wear, fatigue, thermal cycling, end-of-life margin | Not in the RFI |
| Environmental drift | Behavior preserved across the operating environment's range (sea state, visibility, weather, temperature), not only nominal conditions | Not in the RFI |
| Calibration band | Behavior preserved within sensor/actuator tolerance achievable in field maintenance | Not in the RFI |

**Why the extension is required.** Thrust (ii) promises "functional equivalence" across variants. Functional equivalence is a property of a model. An autonomous platform spends most of its service life away from nominal. If the invariant set does not include the degradation envelope, two variants can be proven functionally equivalent and behave differently in the field, and the difference will be discovered by the adversary, not by the assurance process.

The distinction between environmental conditions and degradation matters. Environmental conditions (sea state, visibility, wind, temperature) are named in the regulation; see below. Wear, fatigue, thermal cycling, and end-of-life margin are not named in the regulation and are the harder of the two extensions to require. Both belong in the invariant set, but they enter through different justifications.

**The maritime regulation covers environmental conditions, not degradation.** COLREGs Rule 6 (Safe speed) names, as factors to be taken into account: visibility; traffic density; maneuverability; background lights; state of wind, sea, and current; draft, and, for radar-equipped

vessels, radar characteristics, range scale, sea-state effects on radar, possibility of undetected small vessels, and radar-based visibility assessment. Rule 19 (Restricted visibility) requires speed reduction when a fog signal is heard forward of the beam. These rules cover environmental conditions in prose. They do not cover wear, fatigue, thermal cycling, or end-of-life margin. The §5 extension is therefore partly a formalization of what the regulation asks for in prose (environmental drift), and partly an addition the regulation does not address (degradation envelope, calibration band).

An invariant list functions as a review checklist. What is not on the list is not checked. An implicit invariant is an invariant no one is responsible for verifying.

**The claim.** An invariant set that omits degradation certifies the system that was tested, not the system that is deployed.

## §6. DESIGN DEGREES OF FREEDOM

*[RFI 1(v), Thrust (i)]*

Thrust (i) requires exposing “valid design degrees of freedom.” The word doing the work is valid, and the RFI does not define it. A degree of freedom is valid only if varying it provably preserves the §5 invariant set.

| Class | Examples | Validity test |
|---|---|---|
| Implementation DOF | Language, data structure, control-flow organization, scheduling within timing budget | Provable by equivalence checking against the reference model, for the model’s scope |
| Architectural DOF | Sensor suite composition, redundancy topology, communication graph, compute placement | Requires the invariant set at the architectural level; equivalence checking alone does not reach it |
| Physical DOF | Materials, tolerances, mounting, thermal path, actuator selection | Model must include physical dynamics; §5’s degradation envelope becomes load-bearing |
| Algorithmic DOF | Estimation, planning, and control strategy at equivalent performance | Equivalence is behavioral; requires cross-variant testing, not proof alone |

### 6.1 Semantic heterogeneity

The RFI asks for “semantically heterogeneous” CPS. Two variants are semantically heterogeneous when they differ in which failures are system-failing; they are syntactically heterogeneous when they differ in appearance but share the same failure structure. The apparent-versus-genuine DOF distinction is the semantic-versus-syntactic distinction applied to a single DOF. The metric in §7 extends the distinction from a single DOF to the whole family. §4.6’s Scene 4 is the family-level case: a fleet whose apparent diversity masks a shared hidden centralization is syntactically heterogeneous and semantically homogeneous.

### 6.2 The distinction that matters

Two kinds of variation look like heterogeneity and are not:

- **Apparent DOF**: variation that changes the artifact without changing any failure node, or changes it in a way that does not require independent adversary work. Different code, same structure. Adversary knowledge transfers unchanged. Scene 4 in §4.6 is the fleet-level case.
- **Genuine DOF**: variation that changes which nodes are system-failing and whose exploitation requires independent adversary work per variant. This is what reduces transfer.

**The test.** Vary the DOF, decompose both variants to their critical failure nodes, and compare. If the critical node set is unchanged, the DOF is apparent. If it changed, ask the follow-up: does the exploitation technique developed against variant A give material advantage (at least $\theta$, §7.2) against variant B? If yes, the DOF is apparent at the adversary's cost level even if not at the artifact level. If no, the DOF is genuine. §7.2 operationalizes this at the metric level.

**The boundary.** Diversity is in out-of-envelope failure, not in-envelope behavior. Within the §5 invariant set, all variants must behave equivalently. Diversity in failure modes means: given a stress outside the declared envelope, variants fail differently. Diversity in in-envelope behavior violates the invariant, not the heterogeneity requirement. Exploits that act through specified in-envelope behavior apply to every variant (§4.6, mechanism (j); §12, L10).

**The point.** The RFI asks for degrees of freedom; the useful question is not how many exist but which ones move a failure node in a way that moves the adversary's work.

### 6.3 Illustration: COLREGs compliance as a heterogeneity problem

The COLREGs (International Regulations for Preventing Collisions at Sea, 1972) are the global rules of the road for vessels: 41 rules in six parts (A–F) plus four technical annexes, adopted 20 October 1972, in force since 15 July 1977, amended in 1981, 1987, 1989, 1993, 2001, 2007, and 2013. The framework's categories map onto the regulation's structure.

| COLREGs provision | What it requires | Framework category |
|---|---|---|
| Rule 5 (Look-out) | "a proper look-out by sight and hearing as well as by all available means appropriate in the prevailing circumstances and conditions" | A specification with genuine DOF |
| Rule 6 (Safe speed) | Account for visibility, traffic density, maneuverability, background lights, wind/sea/current, draft, and radar characteristics | Environmental conditions: the part of §5 that the regulation does cover |
| Rule 7 (Risk of collision) | "Assumptions shall not be made on the basis of scanty information, especially scanty radar information" | Analogy to residual declaration (§8.3). The regulation names a concern; it does not specify a residual-declaration procedure. |
| Rule 8 (Action) | "positive, made in ample time… large enough to be readily apparent to another vessel observing visually or by radar; a succession of small alterations… should be avoided" | Analogy to explanation faithfulness (§10.5). The regulation requires legibility to another vessel; it does not specify how legibility is verified. |
| Rule 8(f) (Not to impede) | A vessel required not to impede another's passage must take early action and remains subject to other rules | Compound obligation: one invariant is not dischargeable by satisfying another |

| COLREGs provision | What it requires | Framework category |
|---|---|---|
| Rule 17(a)(ii) (Stand-on, permissive) | May act “as soon as it becomes apparent to her that the vessel required to keep out of the way is not taking appropriate action” | Time-dependent permissive threshold with an undefined trigger |
| Rule 17(b) (Stand-on, mandatory) | Shall act when “collision cannot be avoided by the action of the give-way vessel alone” | Mandatory threshold switch; the trigger is an inference, not an observation |
| Rule 19 (Restricted visibility) | Reduce speed when visibility is restricted | Environmental conditions: the part of §5 that the regulation does cover |
| Rule 2 (Responsibility) | Depart from the rules only to avoid “immediate danger” | Licensed deviation; authority question |
| Rule 28 (Inland vs. International) | International Rule 28 prescribes lights for vessels constrained by draught; Inland § 83.28 is [Reserved] | Specification plurality at the same rule number |
| Amendment history | Seven amendments since 1977 | Specification plurality across time |
| Part F (Verification) | IMO Member State Audit Scheme; Rule 41: Contracting Parties subject to periodic audits | Verification at the wrong granularity |

**The specification and its DOF.** Rule 5’s requirement admits many implementations (radar, EO/IR, acoustic sensing, LIDAR, AIS fusion, and combinations), and each implementation is a variant in the §6 sense. The clause “appropriate in the prevailing circumstances and conditions” is what makes the DOF real: without it, the rule would demand all means absolutely; with it, the rule makes context-bounded choice legitimate.

**Apparent versus genuine DOF, illustrated.** Two USVs with different sensor suites that degrade in the same sea state under the same failure mode are apparently diverse: different hardware, same failure node. An adversary who has developed an exploit against one has developed it against both. Two USVs with suites that fail under different sea states are genuinely diverse, provided the exploitation technique for one does not transfer to the other.

**Two thresholds in Rule 17, of different types.** Rule 17(a)(ii) is permissive: the stand-on vessel may act when it “becomes apparent” the give-way vessel is not acting. Rule 17(b) is mandatory: shall act when “collision cannot be avoided by the action of the give-way vessel alone.” Different autonomy stacks will set both thresholds differently. If either threshold is not in the invariant set, two variants can both be compliant and behave differently in the same encounter.

**Rule 8(f) builds a compound obligation.** A vessel required not to impede another’s passage must take early action and remains subject to other rules. Satisfying Rule 15 does not discharge Rule 8(f); satisfying Rule 8(f) does not discharge Rule 15. The compliance claim for an unmanned vessel is a set of simultaneously-held invariants, not a single behavioral output.

**Rule 2 is a licensed deviation.** For a manned vessel, the master’s judgment is the authority. For an unmanned vessel, the departure decision is made by the autonomy stack. Different stacks will have different departure thresholds. If those thresholds are not declared and bounded, two

compliant variants can behave differently under the same danger.

**Specification plurality is spatial and temporal.** Inside the U.S. COLREGs demarcation lines, nearly identical Inland Navigation Rules (33 CFR Part 83) apply, with specified differences. The clearest is Rule 28: International Rule 28 prescribes lights for vessels constrained by their draught; Inland § 83.28 is [Reserved]: the rule number exists in both regimes, but one carries a rule and the other does not. The plurality is also temporal: the regulation has been amended seven times. A design family whose assurance artifact was proven against one version of the rule may not satisfy a later version. The §11 versioning requirement for reusable formalization libraries is the same problem in program form.

**Verification at the wrong granularity.** Part F of the COLREGs verifies compliance through the IMO Member State Audit Scheme, at the flag-state level (Rule 41: Contracting Parties subject to periodic audits), not the vessel level. The operational compliance claim ("this vessel kept a proper lookout") is made at a granularity the verification regime does not reach. This is the §8.3 residual in institutional form.

**The illustration's scope.** The COLREGs case shows the framework's categories apply to a regulatory specification. It does not show that the framework's central claim (that heterogeneity raises the adversary's acquisition cost under $A_1$ and $A_2$-hybrid) is true for USV/UUV fleets. That claim requires an adversary, a variant family, a measured $H_{d,variant}$, and a §9 protocol run.

## §7. HETEROGENEITY METRIC: VARIANT INDEPENDENCE

*[RFI 2(i)]*

**Subject.** A variant family $F = \{v_1 \dots v_n\}$ generated from one specification.

**Scale.** Two related quantities: a node ratio ($H_{d,node}$) and a variant fraction ($H_{d,variant}$). §4.1's formula uses $H_{d,variant}$. The node ratio is the diagnostic; the variant fraction is what the formula prices.

**Purpose.** To distinguish variation that changes which failures are system-failing and requires independent adversary work per variant from variation that does not.

### 7.1 Why surface metrics fail

Three surface metrics count differences that do not change the adversary's problem.

| Surface metric | What it counts | Why it is insufficient |
|---|---|---|
| Variant count | n members in the family | n identical failure structures is one target, not n |
| Code / implementation diversity | Textual or structural difference | Differences that leave failure nodes unchanged transfer adversary knowledge unchanged |
| Failure-mode count | Distinct failure modes observed | Modes can differ in detail while the system-failing node is common or while the exploitation technique transfers |

Published diversity measures quantify failure correlation or code-level variation: Littlewood and Miller [R4] model the probability of coincident failure across versions, and Larsen et al. [R5] survey automated software diversity. $H_{d,variant}$ counts the variants whose cheapest exploit path is independent, which is the quantity the acquisition-burden formula prices.

### 7.2 The metric: two quantities, one formula

For each variant v, decompose to critical failure nodes at a declared depth d: components whose failure produces system-level failure within the declared model scope, weighted by consequence severity. Let $N_c(v, d)$ be the critical failure node set for variant v at depth d.

The metric measures the adversary's cost of finding and exploiting critical nodes within the declared model scope, not the probability of failure from environmental stress. Out-of-envelope failure (§6.2) is a separate property (it is what makes the variants behaviorally diverse under stress) and is addressed by the invariant set in §5, not by $H_{d,variant}$.

**Three analyst-judgment points.** The procedure below contains three points at which the analyst's reading determines the result. Each is resolved to reduce analyst-dependence, and the residual dependence is declared in §12, L9. Without these resolutions, two analysts applying the procedure to the same pair can produce $H_{d,variant}$ values that differ by the full range (0 to 1 for a two-variant pair). The resolutions below reduce that divergence; they do not eliminate it, and its magnitude is measured in the pilot (§12), because a formal definition of "adversary effort in hours" would require threat-intelligence inputs the framework does not have.

**J-1: node granularity.** By default, a node is a specific artifact: a component with a part number and firmware version. The radar in variant V1 is not the same node as the radar in variant V2 even if both are "X-band radar" of the same functional class. Functional-class granularity (treating "radar" as one node across variants) is permitted as an explicit alternative; it classifies more nodes as common and gives a lower $H_{d,variant}$. The analysis report must state which granularity it used. The default is specific-artifact because an exploit developed against one part number and firmware version does not automatically apply to another. The specific-artifact reading classifies fewer nodes as common, which raises $H_{d,variant}$ and the formula's predicted adversary burden, so it can overstate the defender's benefit (§7.3 lists the directions of error).

**J-2: generic vs. specific transfer.** Generic transfer does not count. A technique that any adversary could have developed without capturing any variant is background knowledge, not transfer. Only a technique developed against a specific variant and shown to give material

advantage against another variant counts as transfer. This makes the material-advantage threshold testable: the analyst compares the cost of applying V1's variant-specific technique to V2 against the cost of a generic attack on V2. If V1's technique is materially cheaper (by at least θ, defined below), transfer exists; if the cost is unchanged, it does not. This resolution also closes mechanism (d) of §4.6's Scene 4 at the node level: a generator's low-entropy choices are background knowledge if any adversary could infer them from the family's public specification, and are transfer if they can only be inferred from a captured design. The distinction is operationalized by the same cost comparison.

**J-3: cheapest path in what unit.** Step 3's "least adversary work" is measured in expected adversary effort in hours for a declared adversary capability class named in the analysis report. The cheapest path has the lowest expected hours. Paths within 20% of each other are declared co-cheapest, and the variant's classification depends on both: if either co-cheapest path is common, the variant is non-independent; if both are variant-specific, the variant is independent. This rule lowers $H_{d,variant}$. The 20% figure is a design choice declared here; alternative bands are permitted if declared.

**Step 1: adversary-independence test.**
For each node X in $N_c(v, d)$ that does not appear in $N_c(v', d)$ for any other variant v', ask: does a technique developed against X in v (per J-2) give material advantage against any node Y in any other variant v'? If yes, X is counted as common, because the adversary's work on X transfers. If no, X is counted as variant-specific. This is the operationalization of §6.2's apparent-versus-genuine distinction at the node level. §4.6's Scene 4 is the fleet-level case of what happens when the test is skipped.

**Material-advantage threshold θ.** "Material advantage" is operationalized as a reduction in the adversary's cost of exploiting a variant by at least a declared threshold θ (e.g., 20%). A transfer that reduces cost by less than θ is not counted as transfer. The threshold is declared in the analysis report and is a design choice. The threshold makes the test binary: either the advantage reaches θ (transfer) or it does not (variant-specific). A continuous measure of advantage can inform θ but not replace the binary classification, because $H_{d,variant}$ requires a binary classification.

**Partial-sharing nodes.** A node that appears in some variants but not all is tested against each variant that does not contain it. If the exploitation technique for the node in one variant gives material advantage against a variant that does not contain it (transfer via pattern rather than node), the node is counted as common because the adversary's work is not wasted. If the exploitation gives no material advantage against any variant other than the ones that already contain the node, it is counted as variant-specific for the variant containing it. The independence

test therefore operates on (node, variant) pairs, not on nodes globally.

Common critical nodes $C_c$ = nodes that appear in every variant, plus variant-specific nodes whose exploitation transfers above θ to any other variant.

Variant-specific critical nodes $S_c$ = variant-specific nodes whose exploitation does not transfer above θ to any variant other than those that already contain it.

**Step 2: node ratio (diagnostic).**

$$H_{d,node}(F) = |S_c| / |\cup_v N_c(v, d)|$$

$H_{d,node}$ ranges from 0 (every variant fails at the same nodes, or different nodes with transferable exploitation) to 1 (each variant's critical failures are independently exploitable at the node level). $H_{d,node}$ is a diagnostic, not what the formula uses. A variant with 9 variant-specific nodes and 1 shared node scores high on $H_{d,node}$ even though the shared node gives the adversary a cheap path to every variant. §7.3 works this case.

**Step 3: minimum-path reduction (per-variant).**

For each variant v, identify the cheapest critical path: the node in $N_c(v, d)$ whose exploitation requires the least expected adversary effort in hours (per J-3), where effort is the per-node exploit development effort less the transfer discount from the study of any single other variant (the largest discount applies; cumulative discounts from studying several variants are not counted, §7.3). Classify the variant:

- **Independent**: the cheapest critical path is a variant-specific node (no other variant's study gives material advantage against it).
- **Non-independent**: the cheapest critical path is a common node (some other variant's study gives material advantage).

**Step 4: variant fraction (what the formula uses).**

$$H_{d,variant}(F) = |\{v \in F : \text{cheapest critical path of v is variant-specific}\}| / |F|$$

$H_{d,variant}$ is the fraction of variants whose compromise requires independent adversary work. This is the $H_{d,variant}$ in §4.1's formula. A variant whose cheapest path is a shared node contributes 0 to $H_{d,variant}$ even if it has many variant-specific nodes, because the adversary takes the cheapest path. A variant whose cheapest path is variant-specific contributes 1 even if it also has shared nodes, because the adversary must still do the independent work to reach that variant's cheapest path.

**Family-level generator check.** Mechanism (d) of §4.6's hidden centralizations is not a node-level mechanism: a shared generator whose low-entropy choices point the adversary at the family's failure surface without requiring a capture. J-2's generic-vs-specific transfer resolution handles the node-level case. The family-level check handles the case where no single node carries the transfer but the family as a whole is design-predictable. The check is: if the family's designs are generated by a process whose biases are inferable from a small sample of designs, the family is non-independent regardless of its node-level $H_{d,node}$. If the check finds the family non-independent, $H_{d,variant}$ is reported as 0 for the family. This is a required human judgment, not a computable metric, and is part of the residual analyst-dependence in §12, L9.

Step 1 operates on nodes. Specification-level transfer and transfer through specified in-envelope behavior (§4.6 mechanisms (e) and (j)) are outside $H_{d,variant}$ (§12, L10).

**Why two quantities.** $H_{d,node}$ measures how differentiated the fleet's failure surface is at the node level. $H_{d,variant}$ measures how differentiated the fleet's adversary cost is at the variant level. These diverge when the cheapest-path reduction classifies a variant differently from the node-ratio diagnostic: for example, a variant with many variant-specific nodes but a cheap shared path ($H_{d,node}$ overstates differentiation) or a variant with a variant-specific cheapest path but most nodes shared ($H_{d,node}$ understates). The formula's quantity is $H_{d,variant}$ because the formula prices the adversary's per-variant work; a variant's work is its cheapest path, not its node profile. Reporting both makes the divergence visible: a fleet can be node-diverse and variant-homogeneous, and the formula correctly prices it as variant-homogeneous.

**What $H_{d,variant}$ measures and does not measure.** $H_{d,variant}$ measures variant-level differentiation, not robustness. A family with high $H_{d,variant}$ may consist of variants that are individually fragile in uncorrelated ways: variant A has unique cheapest path X, variant B has unique cheapest path Y, and neither X nor Y appears in the other. That is what FLASH asks for: a fleet whose compromise requires independent effort per variant. A family with low $H_{d,variant}$ is one in which the variants' cheapest paths transfer. Robustness is a separate property, addressed by §5's invariant set.

**Why criticality, independence, and minimum-path are all in the definition.** Counting all failure nodes would be gameable: a variant with a unique cosmetic component that can fail catastrophically would score high even though the component is trivially exploitable. Restricting to critical nodes addresses consequence. The independence test addresses transferability. The minimum-path reduction addresses the cheapest-exploit property: a variant is not defended by having many expensive unique paths if it has one cheap shared one.

### 7.3 The approximation, declared, and why the two quantities diverge

$H_{d,variant}$ is not H. H is defined over the true critical failure set, which is not fully computable for a real cyber-physical system. $H_{d,variant}$ is a computable approximation:

Computed by hierarchical FMEA-style decomposition to a declared depth d, applying the independence test and minimum-path reduction at each level. The decomposition procedure: (1) identify the system-level function; (2) decompose to subsystems whose failure produces system-level failure; (3) for each subsystem, decompose to components at depth d; (4) apply the independence test to each variant-specific node; (5) apply the minimum-path reduction to each variant; (6) aggregate to $H_{d,variant}$.

**Resolution declared:** d is stated; nodes below d are unresolved and counted as common unless analysis resolves them. This is a conservative choice: it understates $H_{d,variant}$ rather than overstates it.

**Direction of error:** $H_{d,variant}$ is not a one-directional bound on $H_{variant}$. Two conventions lower it: unresolved nodes below depth d are counted as common, and J-3's co-cheapest rule classifies a variant as non-independent if either path is common. Three conventions can raise it: J-1's specific-artifact default, the binary threshold θ (a transfer below θ counts as none, and discounts below θ are not cumulated across variants or studies), and a false no-transfer call in Step 1. The report states the conventions used and, where the direction is known, the direction of the resulting error.

**Criticality criterion declared:** the consequence threshold that qualifies a node as critical is stated, with the threshold itself as a design choice.

**The worked example that separates the two quantities.** Consider N = 10 variants, each with 1 shared critical node and 9 variant-specific critical nodes.

- Union of critical nodes: 1 shared + 9×10 variant-specific = 91 nodes.
- Variant-specific critical nodes $S_c = 90$.
- $H_{d,node} = 90/91 \approx 0.99$.

Now apply the minimum-path reduction. For each variant, is its cheapest critical path the shared node or one of the 9 variant-specific nodes? If the shared node is cheapest (e.g., the shared node requires one exploit that transfers), then for every variant the cheapest path is shared.

- $H_{d,variant} = 0/10 = 0$.

The two quantities diverge by the full range. $H_{d,node}$ says the fleet is 99% differentiated; $H_{d,variant}$ says it is 0% differentiated at the adversary's cost level. The formula's prediction $[1 + 0.99 \cdot (10-1)] \times (c_{capture} + c_{exploit}) \approx 9.9 \times (c_{capture} + c_{exploit})$ is wrong for this fleet; the correct prediction is $[1 + 0 \cdot (10-1)] \times (c_{capture} + c_{exploit}) = 1 \times (c_{capture} + c_{exploit})$, because one shared-node exploit compromises all ten variants. The formula must use $H_{d,variant}$, not $H_{d,node}$.

**The bridge from FMEA to adversary cost.** FMEA uses "failure nodes": components whose failure produces system failure. The adversary's cost depends on "exploitable surfaces": components that can be reached, influenced, or bypassed by an attacker. These overlap but are not identical. The metric treats failure nodes as a proxy for exploitable surfaces. The proxy is asserted, not proven, and its validity is a per-domain question. §9's protocol is how the proxy is tested.

### 7.4 The derived measure: adversary acquisition cost

C(A, F) = expected cost for adversary class A to acquire working instance-specific knowledge of a deployed member of F.

**The structural claim, per §4:** C increases with $H_{d,variant}$ under $A_1$ and $A_2$-hybrid (adversary must repeat public study or capture-and-exploit for each variant whose cheapest path is independent); is flat in $H_{d,variant}$ under $A_2$-pure (acquisition is per engagement regardless). The structural multiplier $[1 + H_{d,variant} \cdot (N-1)]$ is the same for $A_1$ and $A_2$-hybrid; the difference is the per-variant unit cost.

**The limitation.** C(A, F) is not computable in closed form. T from §9's protocol estimates the exploit-development component, and the capture component is estimated per §2.1 and §9.3; $H_{d,variant}$ is the structural predictor. The correspondence between $H_{d,variant}$ and the §9 estimate is the hypothesis under test, not a result. $H_{d,variant}$ is a metric; C is a hypothesis with a measurement procedure.

### 7.5 Independent check of $H_{d,variant}$

§9 estimates C by measuring time-to-compromise and checking whether it tracks $H_{d,variant}$. That is circular if $H_{d,variant}$ has no independent validation: if T does not track $H_{d,variant}$, the metric and the protocol fail together.

**The independent check:** compute $H_{d,variant}$ on variant families whose failure behavior is already documented from field history.

- A family where a known failure recurred across variants should have low $H_{d,variant}$. If the computation returns low $H_{d,variant}$, the metric is consistent.

- A family where failures were variant-specific should have high $H_{d,variant}$. If the computation returns high $H_{d,variant}$, the metric is consistent.

The check validates whether $H_{d,variant}$ tracks the documented recurrence of failures across variants, not whether failure-node structure is a valid proxy for exploitable surface. A family with high failure recurrence across variants would be expected to have low $H_{d,variant}$ because recurrence indicates shared structure. The check confirms or refutes the recurrence-to-$H_{d,variant}$ correspondence. The failure-to-exploit bridge (Assumption 3) is tested only by §9's protocol.

This check requires a domain with documented cross-variant failure history. §12 L4 declares that the UUV/USV domain may not satisfy this requirement: compliance-adjudication history is sparse, and field-failure history is not comprehensively public. The check is available for any domain where the history exists; whether UUV/USV qualifies is the first thing a pilot would verify. If the domain does not qualify, the metric's independent check can run against published multiversion experiments with recorded coincident failures (Knight and Leveson [R3]) or against synthetic families with injected, documented failure correlations; the correspondence to real field data is then deferred to §9's protocol.

## §8. VALIDATION OF EQUIVALENCE AND INVARIANTS

*[RFI 2(i), Thrust (ii)]*

### 8.1 Functional equivalence: of what model, by what method

Thrust (ii) requires "guaranteeing functional equivalence and formally provable correctness." A proof establishes a property of a model. Three scopes, each requiring different evidence and different validation methods:

| Model scope | What is proven | Validation methods |
|---|---|---|
| I/O-equivalence over a specification | Each variant computes the same function | Differential fuzzing; test-vector conformance; formal equivalence checking against the reference model; property-based testing over the I/O contract |
| + control and timing | Same function under stated timing and state constraints | Timed-automaton equivalence; statistical model checking over timing properties; hardware-in-the-loop testing |
| + physical dynamics | Same function within a stated dynamic envelope | Co-simulation with validated plant models; residual estimation against field data; envelope testing at declared boundaries |

Equivalence at scope 1 is achievable with current formal methods. Equivalence at scope 3 is where the assurance problem lives, and where §5's degradation envelope, environmental drift, and calibration band enter as required invariants rather than as optional additions.

### 8.2 Other invariants

For each invariant in §5 beyond function, validation has a different shape: safety requires reachability analysis over the hazard set including degradation states; security requires the §2 three-class adversary model; timing and performance require bounds that shift with wear; interoperability is the only invariant whose statement demands cross-variant evidence, and it is also the channel by which Scene 5's topology-adjusted $c_{capture}$ matters.

### 8.3 The model–reality residual

The validation gap is not proof error. It is model–reality mismatch, carried as a residual, not eliminated. The defensible claim for each variant is: equivalent within the stated model scope, with model–reality mismatch carried as a declared residual whose magnitude is the extrapolation beyond the envelope within which the model was validated.

The residual is a quantity that gets declared, so that the assurance process records where the proof stops being a statement about the system and starts being a statement about the model. Proof beyond the model scope is a claim no formal method delivers; a declared residual is what a formal method delivers.

**The maritime regulation names the concern, not the procedure.** COLREGs Rule 7 provides that "assumptions shall not be made on the basis of scanty information, especially scanty radar information." But "scanty" is undefined, and the compliance framework has no mechanism for declaring how much information is enough. The residual declaration in §8.3 formalizes the concern the rule raises; the rule does not itself specify a residual-declaration procedure.

## §9. EVALUATION CRITERIA

*[RFI 2(i), 1(iv)]*

The RFI asks for "evaluation criteria capable of achieving the FLASH vision." The criteria come first; the protocol that operationalizes them follows; the statistical specification comes last.

### 9.1 The six criteria

A heterogeneity claim is assessable if and only if it states:

1. The variant family and its declared critical failure node set, with the three analyst-judgment resolutions of §7.2 applied (J-1 node granularity, J-2 generic-vs-specific transfer, J-3 cheapest-path unit) and the adversary-independence test and minimum-path reduction run.
2. The measured $H_{d,variant}$, the analysis depth at which it was computed, and the analyst's declared range (§7.2, §7.3, §12 L9).
3. The adversary class ($A_1$, $A_2$-hybrid, or $A_2$-pure) against which the claim is made, with the observation and capture model operationally specified (§2, §9.3).

4. The time-to-compromise-f measured under a calibrated autonomous exploit-development agent, with the fraction f declared (f = 1 for full-fleet compromise) and the statistical specification of §9.4.
5. The correspondence between $H_{d,variant}$ and time-to-compromise-f, stated as a test with interpretation fixed in advance (§9.4).
6. The defender's cost: per-variant verification effort and the effort of computing $H_{d,variant}$, recorded in the same experiment (§9.2).

A claim that does not supply all six is not assessable.

## 9.2 The protocol

The RFI asks for experiments "assessing resilience against adaptive, including autonomous exploit development using intelligent agents." The protocol:

**Capability calibration.** Run the autonomous exploit-development agent (capable of adaptive search, hypothesis revision, and independent operation) against a known-vulnerable single-variant baseline, named in the experiment report. The calibration requires ≥5 runs; the agent must achieve a working exploit within a stated budget $T_{cal}$ in all calibration runs. If it cannot, the run is void.

Calibration shows that the agent can find a known exploit within the budget. It does not show that T corresponds to the effort of a human or state adversary; T is reported as agent time and mapped to adversary cost only under the declared capability class (J-3).

**Baseline family.** Deploy a single-variant family. Run the agent until a working exploit is found. Record $T_{f1}$ as the time to compromise the declared fraction f of the fleet.

**Heterogeneous family.** Deploy a family with measured $H_{d,variant}$. Run the same agent class against the fleet. Record $T_{f,N}$ as the time to compromise f·N variants. For the formula's prediction to match, f = 1; if f < 1, the effective N in the formula is f·N, not N, and the experiment report states f. The §4.5 toy computation uses f = 1.

**Sweep $H_{d,variant}$.** Repeat across families from $H_{d,variant} \approx 0$ to $H_{d,variant}$ high.

**Capture cost.** T measures exploit-development time after the declared channel is granted. $c_{capture}$ is set by the channel declaration in §9.3 and estimated separately (§2.1); T alone does not measure $c_{capture} + c_{exploit}$.

**Defender cost.** Record per-variant verification effort and the effort of computing $H_{d,variant}$ for each family.

**Three adversary conditions.** Run under $A_1$, $A_2$-hybrid, and $A_2$-pure, defined operationally below.

## 9.3 Observation and capture model

| | $A_1$ | $A_2$-hybrid | $A_2$-pure |
|---|---|---|---|
| **Sees** | Specification; public documentation; one reference variant | Deployed instance through a declared capture channel, with time to study | Deployed instance through a declared real-time observation channel, without study |
| **Channel** | None into the deployed instance | Named explicitly: physical capture duration, firmware extraction path, software intrusion path | Named explicitly: live probing, side-channel access, no persistent access |
| **Withheld** | Deployed instance state; reference variant differs from deployed | Instance state after capture-and-study window | Instance state beyond live observation window |
| **Topology** | n/a | Declared: how many variants per capture (Scene 5's $k_{cap}$), which sets the number of captures | n/a |

The channel is declared per experiment. An experiment that does not state its channel does not produce a class-attributable result. An $A_2$-hybrid experiment that does not state its capture topology does not produce a topology-attributable $c_{capture}$.

## 9.4 Statistical specification

The protocol requires:

**Run count:** minimum 5 runs per ($H_{d,variant}$, adversary-class) cell. Below 5, the result is reported as preliminary, not as support or falsification.

**Effect-size threshold:** the prediction is multiplicative. For each $H_{d,variant}$ level, the predicted ratio is $T_f(H_{d,variant}) / T_f(H_{d,variant} = 0) = \lceil 1 + H_{d,variant} \cdot (N-1) \rceil$ (with f = 1). Support requires that the 95% confidence interval of the observed ratio excludes 1 and lies within a declared tolerance of the prediction (a design choice, for example a factor of 2). The interval is computed from the runs in each cell. Outside the tolerance the outcome is reported as no support, and the observed effect is reported at its measured size.

**Stochasticity handling:** report mean and 95% confidence interval. Exploit-development search times are right-skewed; the report must state whether the mean, median, or geometric mean is quoted.

**Baseline definition:** the "known-vulnerable single-variant" baseline is a specific single-variant family with a documented exploitable path, named in the experiment report.

**Analyst-dependence declaration:** the analysis report states the $H_{d,variant}$ value used and the range the analyst considered (§12 L9). The formula value is reported at both ends of the range.

**Interpretation:**

| Observation | Interpretation |
|---|---|
| T rises with $H_{d,variant}$ under $A_2$-hybrid, flat under $A_2$-pure ($A_1$ not run) | Hypothesis supported for $A_2$-hybrid; the structural asymmetry confirmed |
| T rises with $H_{d,variant}$ under $A_1$ and $A_2$-hybrid, flat under $A_2$-pure | Hypothesis supported for both classes; the structural multiplier is shared |
| T flat under $A_1$, rises under $A_2$-hybrid | Structural-symmetry claim in §4.1 falsified |
| T rises under $A_1$, flat under $A_2$-hybrid | Hypothesis falsified for $A_2$-hybrid; the §4 argument does not hold where it claims to pay |
| T flat under all three classes | Hypothesis falsified; $H_{d,variant}$ is not predictive |
| T rises under all three | Hypothesis falsified; heterogeneity imposes cost under $A_2$-pure, which §4 did not predict |
| T flat under $A_2$-hybrid, rises under $A_2$-pure | Hypothesis falsified in the opposite direction |
| $T_{cal}$ not met in calibration | Run void; no result |

### 9.5 Formalizable structure

The agent's search is a structure: the observation and capture model (§9.3), the operation set (what the agent can do), and the termination condition (a working exploit). The experiment tests a stated structural hypothesis. The empirical run estimates parameters; the structure makes the estimate interpretable.

### 9.6 Formal versus empirical

§9 is an empirical protocol. The formal component of this response is §7's $H_{d,variant}$ procedure, a structured computation with three analyst-judgment points (L9), not a proof. It makes heterogeneity measurable by a stated procedure, which is the precondition for assessing FLASH's central thesis. The metric is a formal construct; its correspondence to adversary cost is tested empirically in §9.

### 9.7 Declared status

The protocol is proposed, not yet run. No experiment has been performed. §4.5's toy computation checks the argument's arithmetic, and §4.6's scenes restate that check spatially.

# §10. SYNTHESIS AND ARCHITECTURE

*[RFI 2(ii), 2(iii), Thrust (iii)]*

## 10.1 Where the symbolic layer sits

The RFI's premise (augmenting generative agents with symbolic reasoning to "formally represent design intent") is sound as a division of labor, provided the symbolic layer's authority is specified. Three placements:

| Placement | What the symbolic layer does | What it can and cannot guarantee |
|---|---|---|
| Generator | Symbolic layer produces the design; the model elaborates it | Strongest formal guarantee; weakest generative range |
| Checker | Model produces the design; symbolic layer verifies it | Broadest generative range; verification is per-output and costs scale with n |
| Constrainer | Model generates within a symbolic envelope; the envelope is proven, the interior is not | Middle cost; the guarantee is only as good as the envelope |

The choice determines whether thrust (ii)'s guarantee is available per variant at generation time or only after the fact. The generator is also a potential hidden centralization (§4.6 Scene 4, mechanism d): if the generator makes consistent choices across a family, an adversary who learns the generator's bias gains discovery advantage without a capture. This is a property of the synthesis pipeline, not of the variants themselves, and it belongs in the generator placement's risk profile.

## 10.2 Structurally constrained synthesis

The RFI asks for "structurally constrained induction and deduction." The constraint is a property that makes the generator assurable. An unconstrained generator can propose designs the verifier cannot check at acceptable cost, because verification cost grows with the design's distance from the reference model. A structurally constrained generator proposes only designs within the scope of the reference model, which is the scope the assurance artifact covers (§8.1).

Oracle-guided inductive synthesis is one way to impose the constraint: the oracle rejects candidates outside the scope. Combining a generative multimodal model with an interactive theorem prover is another: the prover's rejection is the constraint, and its acceptance is the assurance. Either way, the structural constraint is what makes assurance tractable per variant.

## 10.3 Single-agent and multi-agent architectures

Single-agent synthesis centralizes the design loop and makes provenance traceable; its failure is the single point of generator bias.

Multi-agent synthesis allows division into proposal, critique, and verification roles; its failure is that the agents may share training lineage and therefore share the bias the separation was meant to address.

The architecture that matters for assurance is not the number of agents but whether the verifying agent is independent of the generating agent in the senses specified below.

## 10.4 The trust-transfer problem, with independence specified

Thrust (iii) calls for co-generated assurance and explanation artifacts. If the same model class generates the design and its proof, the proof is a statement about what that model believes, and the trust that was in the design has moved to the generator rather than been removed.

**Three axes of independence:**

| Axis | What it means | What it gives |
|---|---|---|
| Data independence | Design and assurance models trained on different corpora | Weak; same architecture on different data still shares inductive biases |
| Architectural independence | Different model classes (e.g., a language model generates; a theorem prover checks) | The minimum the requirement needs. A prover cannot fail the way a language model fails |
| Process independence | Different organizations or teams generate design and assurance | Strongest; not always feasible within a program |

**The requirement:** the assurance path must be architecturally independent of the synthesis path at minimum. Process independence is preferred where organizationally achievable. Data independence is insufficient on its own.

**The specification-formalization path.** Architectural independence between design and assurance is necessary but not sufficient. If the specification itself is model-generated, then the proof is of a model-generated spec, and mis-formalization, not proof error, is the failure mode. The full requirement is: architectural independence between design and assurance plus independent validation of the specification. The specification-validation path can be a human domain expert, a domain-specific reference model not generated by the same process, or a formal property check against an established domain standard.

**On agent count.** Multi-agent architectures reduce bias only if the agents are independent in the senses above. Agents drawn from the same model class share lineage, and shared lineage produces correlated failure modes. Prompt-level diversity reduces correlation weakly and unmeasurably.

## 10.5 Explanation and understanding

The RFI requires explanation artifacts alongside assurance artifacts, and architectures suitable for both design and understanding. These are not the same output, and the independence requirement differs between them.

| | Assurance artifact | Explanation artifact |
|---|---|---|
| **What it establishes** | That a property holds | Why a property holds, in terms the reader can follow |
| **Verification** | Proof or test, checked mechanically | Comprehension by a named, non-author checker: a human who can predict system behavior from the explanation, or a derivation from the assurance artifact |

| | Assurance artifact | Explanation artifact |
|---|---|---|
| **Failure mode** | Proof error, or model–reality mismatch | Persuasiveness without faithfulness: the explanation is coherent and wrong |
| **Who is positioned to verify** | A prover | The named checker |

The trust-transfer problem applies with more force to explanation than to assurance. An explanation co-generated by the same model that generated the design explains the model's model of the system, not the system: the same mismatch as §8.3, arriving as narrative rather than as a declared residual. Explanation is optimized for comprehensibility, which correlates with persuasiveness, not faithfulness. A coherent explanation of a wrong design is worse than no explanation, because it removes the reader's motive to look further.

**The criterion, with the checker specified.** Explanation must be architecturally independent of synthesis, and it must be checkable against the assurance artifact, not against the design. The checker is a named party who is not the explanation's author and not the synthesis process. Agreement is checked by one of two methods, declared in the output: (a) derivation: the checker derives the proof's claim from the explanation and confirms it matches the proof; (b) counterexample search: the checker attempts to construct a system that satisfies the explanation but not the proof, or the reverse. If a model is the checker, the model must be architecturally independent of the explanation generator. If a human is the checker, the method and the human's basis are recorded.

**The maritime regulation requires legibility of behavior.** COLREGs Rule 8 requires any alteration of course or speed to avoid collision be "large enough to be readily apparent to another vessel observing visually or by radar," and that "a succession of small alterations of course and/or speed should be avoided." The requirement parallels the explanation-faithfulness criterion: the action must be readable by the other party. The parallel is an analogy; Rule 8 governs the maneuver, not an explanation artifact. §10.5's criterion addresses the concern Rule 8 raises.

**What understanding buys the program.** A design whose assurance artifact no human can follow is a design whose operators cannot recognize when it is behaving outside its proven scope. Understanding is the channel through which the assurance's residual reaches deployment.

## §11. REUSABLE FORMALIZATION LIBRARIES

*[RFI 2(iv)]*

What makes a formalization reusable across variants is not its subject matter but its level of abstraction relative to the degrees of freedom in §6.

| Content | Reusable across variants? | Why |
|---|---|---|
| Specification of required behavior | Yes | It is what equivalence is measured against |

| Content | Reusable across variants? | Why |
|---|---|---|
| Reference model at a chosen scope | Yes, within the scope | The same model serves as the equivalence target |
| Proof of the reference model | Yes | The proof is of the specification, not the implementation |
| Failure-history corpus for the domain | Yes | The input to §7.3's decomposition; documented failures do not depend on the variant |
| Proof tactics and lemma libraries | Partially | Structurally similar variants reuse proof machinery heavily; the machinery is reusable, the final proof script is not |
| Per-variant proof content | No | The proof is of the implementation |
| Per-variant test suite | Partially | Specification-level tests are reusable; instance tests are not |

**What the library contains, structurally:** a specification, a reference model with declared scope, the proof of the reference model, and a failure-history corpus. Per-variant artifacts are generated against it, not stored in it. A library that stores per-variant proofs has stored the variants, not the reusable structure.

**Versioning.** The reference model's scope is versioned because the scope is what changes when the invariant set is extended (§5). A library whose scope is not versioned silently redefines what its members are proven to do. The maritime case shows why: the COLREGs have been amended seven times since 1977, and a design family whose assurance artifact was proven against one version may not satisfy another. A library without scope versioning does not merely fail to track this; it conceals it.

**Source editions.** All COLREGs citations in this response are from the IMO Convention text (Convention on the International Regulations for Preventing Collisions at Sea, 1972, as amended through the 2013 amendment in force 1 January 2016) and 33 CFR Part 83 as published in the eCFR as of 5 November 2026.

## §12. ASSUMPTIONS, LIMITATIONS, AND FALSIFIERS

*[RFI 1(iv)]*

### Assumptions

**Assumption 1: Acquisition condition.** Heterogeneity raises adversary knowledge acquisition cost under $A_1$ and $A_2$-hybrid by the same structural multiplier $[1 + H_{d,variant} \cdot (N-1)]$, at different per-variant unit costs ($c_{public}$ for $A_1$; $c_{capture} + c_{exploit}$ for $A_2$-hybrid). It does not affect the burden under $A_2$-pure for static or slow-dynamic heterogeneity. Which class applies to a program is determined per §2.1.

**Assumption 2: Equivalent model scope.** Variants are generated under a single specification whose scope is declared, not assumed.

**Assumption 3: Measurability of critical failure nodes.** The decomposition in §7 is applicable to the class, not only to individual artifacts. The FMEA-to-adversary-cost bridge (§7.3) is asserted, not proven; its validity is a per-domain question. The formula has no usable input if this assumption fails; §9's protocol is the test (L1).

**Assumption 4: No missed transfer.** The $A_2$-hybrid burden formula applies when the §7.2 test has identified every transfer between variants, including transfers through a common software stack, supply chain, physical weakness, generator, or any other mechanism in §4.6 Scene 4. Transfers the test misses (below θ, family-level, specification-level) are not priced, and the real burden is then closer to the homogeneous case (§4.5, §4.6 Scene 4).

**Assumption 5: $H_{d,variant}$ is measured post-test and post-reduction.** The $H_{d,variant}$ in §4.1's formula is the value produced by §7.2's full procedure: independence test (with material-advantage threshold θ), minimum-path reduction (with J-3's expected-effort-in-hours unit and 20% co-cheapest band), and the three judgment-point resolutions, yielding the variant fraction whose cheapest critical path is variant-specific. A formula evaluated with $H_{d,node}$ (node ratio without minimum-path reduction) overstates or understates the adversary's burden depending on the failure structure: it overstates when a cheap shared path is invisible in the node ratio (§7.3's example). A precondition for interpreting any number the formula produces is that the full procedure has been run.

**Assumption 6: The capture term is topology-adjusted.** The capture term in the formula reflects the deployment topology. Co-located or channel-connected variants may be reachable through a single capture, in which case the number of captures is $\max(1, \lceil \text{cycles} / k_{cap} \rceil)$, where $k_{cap}$ is the average variants-per-capture (§4.6 Scene 5). The formula's cycle count is unchanged; the capture term depends on topology, not only on per-variant capture difficulty.

**Assumption 7: Cluster structure.** The formula assumes the non-independent $(1 - H_{d,variant})$ variants form a single transfer cluster. If they form $k_{clusters} > 1$ transfer-equivalence classes, the cycle count is $H_{d,variant} \cdot N + k_{clusters}$ (§4.5). Reports give $k_{clusters}$ alongside $H_{d,variant}$.

**Limitations**

**L1. Metric validity.** The correspondence between $H_{d,variant}$ and adversary cost is a hypothesis (Assumption 3). §7.5 tests the correspondence to documented failure recurrence. §9's protocol tests the correspondence to adversary cost.

**L2.** §9 is proposed, not run (§9.7).

**L3.** $H_{d,variant}$ is an approximation of $H_{variant}$ with error in both directions. §7.3 lists the conventions that lower it and those that can raise it. The report states the conventions used.

**L4. Anchor basis.** The anchor instance is the UUV/USV class, named from public regulatory documents (COLREGs Convention text; 33 CFR Part 83 as published in eCFR) and general industry practice. The response does not claim program participation, fleet data, or operational experience in the domain. Additional limits: (a) the UUV/USV domain may not satisfy §7.5's requirement for documented cross-variant failure history; (b) item [1(i)] is addressed at the class level only, with no specific platform or program proposed.

**L5.** §3's characterizations of current defenses are a reading of the field. They are position, not measurement. No systematic literature survey was conducted.

**L6. The $A_2$-hybrid formula's boundary for Assumption 4.** The formula applies only when independence holds. §4.5 and §4.6 state the condition under which it fails: shared centralization collapses toward the homogeneous case.

**L7. The formula's dependency on measurement discipline (Assumption 5).** §4.1's formula requires $H_{d,variant}$ as defined in §7.2, which is a post-test-and-post-reduction value. Computing $H_{d,node}$ and using it in the formula can overstate or understate the adversary's burden; the direction depends on the failure structure. §4.6's Scene 3 scope note and §7.3's worked example make the precondition visible.

**L8.** The defender's burden includes the $H_{d,variant}$ computation. §4.1 states it as N × per-variant verification plus the computation. §7.2's independence test and minimum-path reduction are pairwise, so the computation is potentially $O(N^2)$ in the worst case, and each comparison is an adversary-effort estimate of the kind §3 lists as red-team cost. Whether the computation is amortizable across variant families, reducible by heuristics, or a fixed per-family overhead is left to a pilot.

**L9.** $H_{d,variant}$ is analyst-dependent at the three judgment points resolved in §7.2. J-1 (node granularity), J-2 (generic vs. specific transfer), and J-3 (cheapest path in expected hours) are each resolved with a default and an alternative, but the analyst still estimates adversary effort in hours, judges whether a technique transfer meets $\theta$, and judges whether specific-variant techniques are genuinely distinct. Two analysts applying the procedure to the same pair may produce different $H_{d,variant}$ values. §7.2's resolutions reduce the divergence; they do not eliminate it, because a formal definition of adversary effort in hours would require threat-intelligence inputs the framework does not have. The analysis report states the $H_{d,variant}$ value used and the range

considered, with the formula value at both ends of the range. The pilot measures the divergence (see “What a funded next step would be”).

**L10. Specification-level and in-envelope exploitation.** $H_{d,variant}$ is computed over component-level critical nodes. Exploitation through a shared specification or through specified in-envelope behavior (§4.6, mechanisms (e) and (j)) applies to every variant by construction and is not priced by the formula. Under such exploitation the burden is the homogeneous case regardless of $H_{d,variant}$.

**L11. Scope conditions.** Assumption 2 (declared model scope, §8.1, §8.3), Assumption 6 (topology, §4.6 Scene 5), and Assumption 7 (cluster structure, §4.5) bound where the formula applies. Outside them the formula’s cycle count is not a prediction.

**What would falsify this response**

**Flat T under $A_2$-hybrid (§9.4).** If $H_{d,variant}$ does not predict adversary cost under the class where the argument claims it pays, the §4 argument is falsified.

**$H_{d,variant}$ uncorrelated with documented failure history (§7.5).** If $H_{d,variant}$ computed on families with known failure behavior does not track that behavior, the metric is not measuring what §7 claims.

**The $A_1$ structural multiplier is not shared with $A_2$-hybrid (§9.4).** If T rises under $A_2$-hybrid but is flat under $A_1$, the structural-symmetry claim in §4.1 is falsified.

**Program-feasibility conditions, distinct from argument-falsifiers:**

**Architectural independence infeasible at acceptable cost (§10.4).** If no independent assurance path can be built within program constraints, the independence requirement is not a program option.

**Bedrock decomposition not computable at any useful depth (§7.3).** If $H_{d,variant}$ cannot be computed at any useful depth for real systems, the metric is not usable.

**Specification-formalization path not implementable (§10.4).** If the full independence requirement cannot be met, the response claims more than a program can deliver.

**Outcomes that change the practical claim:** $A_1$ dominance in every RFI example keeps the structural multiplier at a lower absolute per-variant cost. $A_2$-pure dominance in the RFI’s examples removes the benefit of static heterogeneity, because that class is unaffected by it. §2.1 determines which class applies to a program; §9 tests the burden under each class.

**What a funded next step would be**

A small pilot on one variant family in a domain where cross-variant failure history is public. The pilot would: compute $H_{d,variant}$ (§7.2, §7.3) with the three analyst-judgment resolutions applied and the $H_{d,variant}$ range declared; record the deployment topology and its $k_{cap}$; run the §9 protocol under $A_1$, $A_2$-hybrid, and $A_2$-pure; test the correspondence between $H_{d,variant}$ and measured $T_f$; and report either a validated metric or a falsified one. The pilot's first task is to verify the domain satisfies §7.5's requirement: that documented cross-variant failure history exists. The pilot's second task is to test analyst repeatability: two analysts independently apply §7.2's procedure to the same variant pair and compare $H_{d,variant}$ values. The divergence between their values is the measured analyst-dependence residual, reported in place of L9's qualitative statement. The pilot also applies §2.1 to the chosen family to determine the dominant adversary class.

## ANNEX A. COVERAGE MAP

Every requested-information item in DARPA-SN-27-17 and where this response addresses it.

| RFI ask | Topic | Section |
|---|---|---|
| 1(i) Operational domain and target application | Applications | §1 |
| 1(ii) National-security rationale and threat environment | Applications | §2 |
| 1(iii) State of the art; limitations; points of failure | Applications | §3 |
| 1(iv) Heterogeneity mitigation; assumptions; limitations; evaluation methods | Applications | §4, §4.3, §4.5, §4.6, §12 |
| 1(v) Functional, safety, security, timing, interoperability, performance invariants; DOF | Applications | §5, §6 |
| 2(i) Metrics for heterogeneity; equivalence validation; adversarial experiments | Frameworks | §7, §8, §9 |
| 2(ii) Structurally constrained induction/deduction; oracle-guided synthesis; generative models + theorem provers | Frameworks | §10.1, §10.2 |
| 2(iii) Single-agent and multi-agent architectures | Frameworks | §10.3, §10.5 |
| 2(iv) Reusable libraries of formalized specifications, models, proofs | Frameworks | §11 |

| Background claim / thrust | Where engaged |
|---|---|
| Augmenting generative agents with symbolic reasoning to formally represent design intent | §10.1 |
| Meaningful variation while preserving specified properties | §4, §5, §7 |
| Decoupling high-level requirements from instance-level design details | §6.1, §8.1 |
| Variation reduces durability and transferability of adversary knowledge | §4 (engaged and reframed via three-class analysis) |
| Common fault modes as adversary knowledge | §4.2 |
| Semantic heterogeneity | §6.1 |
| Correctness, heterogeneity, resilience formally assessed, not solely empirical | §9.6 |
| Dynamic heterogeneity | §4.3 |
| Thrust (i): formalize requirements and expose degrees of freedom | §6 |
| Thrust (ii): guaranteed functional equivalence and provable correctness | §8 |
| Thrust (iii): assurance and explanation artifacts | §10.4, §10.5, §11 |

**Coverage declaration.** All 9 numbered sub-asks (1(i)–(v), 2(i)–(iv)) have an address. All 11 background claims and thrusts are engaged directly: four in dedicated sections (§6, §8, §10.4, §10.5), seven integrated into the argument (§4, §4.2, §4.3, §5, §7, §8.1, §9.6). Item [1(i)] is addressed at the class level with one concrete function (autonomous maritime patrol), as declared in §12, L4.

## ANNEX B. WHAT WOULD FALSIFY THIS RESPONSE

| Falsifier | What it would show | Where observed |
|---|---|---|
| Flat time-to-compromise under $A_2$-hybrid | $H_{d,variant}$ is not predictive of adversary acquisition cost where the argument claims it pays | §9 protocol |
| $H_{d,variant}$ uncorrelated with documented failure history | The metric does not measure what §7 claims | §7.5 independent check |
| The $A_1$ structural multiplier is not shared with $A_2$-hybrid (T rises under $A_2$-hybrid, flat under $A_1$) | The structural-symmetry claim in §4.1 is wrong | §9 protocol |

**Program-feasibility conditions, distinct from argument-falsifiers:**

- Architectural independence infeasible at acceptable cost (§10.4)
- Bedrock decomposition not computable at useful depth (§7.3)
- Specification-formalization path not implementable (§10.4)

**Outcomes that change the practical claim:** $A_1$ dominance, or $A_2$-pure dominance in the RFI's application examples. $A_2$-pure is the class static heterogeneity does not affect; if it dominates the RFI's use cases, the framework's practical claim shrinks.

## REFERENCES

**[R1]** Boston Globe, 16 December 2016, "Pentagon demands that China return seized Navy drone." https://www.bostonglobe.com/news/world/2016/12/16/chinese-warship-stole-navy-underwater-drone-says/BYai60wC73FrBGan95Sj4I/story.html

**[R2]** Marine Log, 20 December 2016, "China returns US drone." https://www.marinelog.com/news/china-returns-us-drone/

**[R3]** J. C. Knight and N. G. Leveson, "An Experimental Evaluation of the Assumption of Independence in Multiversion Programming," IEEE Transactions on Software Engineering, SE-12(1), 1986, pp. 96–109.

**[R4]** B. Littlewood and D. R. Miller, "Conceptual Modeling of Coincident Failures in Multiversion Software," IEEE Transactions on Software Engineering, 15(12), 1989, pp. 1596–1614.

**[R5]** P. Larsen, A. Homescu, S. Brunthaler, and M. Franz, "SoK: Automated Software Diversity," IEEE Symposium on Security and Privacy, 2014.

## POC AND FORMAT BLOCK

| | |
|---|---|
| **Organization:** | AionSystem |
| **Technical POC:** | Sheldon K. Salmon |
| **Email:** | AionSystem@outlook.com |
| **Phone:** | (315) 460-6977 |
| **RFI reference:** | DARPA-SN-27-17 |
| **Classification:** | UNCLASSIFIED. No proprietary information is included in this response. |
| **Format compliance:** | 8.5 × 11 inch pages, 1 inch margins, 12-point body font, 8–10 point font in tables. File format: unprotected PDF. |
| **Supplement:** | interactive simulation; this full paper and the RFI submission version are available for download: https://aionsystem.github.io/H.U.O/index.html |